FAKULTA MECHATRONIKY,
INFORMATIKY A MEZIOBOROVÝCH
STUDIÍ TUL

Diplomová práce

Architektura systému pohledové inspekce kvality

Studijní program: N0613A140028 – Informační technologie
Studijní obor: N0613A140028AI – Aplikovaná informatika
Autor práce: **Bc. Kevin Daněk**
Vedoucí práce: Ing. Igor Kopetschke

Liberec 2025

Tento list nahraďte
originálem zadání.

PROHLÁŠENÍ

Prohlašuji, že svou diplomovou práci jsem vypracoval samostatně jako původní dílo s použitím uvedené literatury a na základě konzultací s vedoucím mé diplomové práce a konzultantem.
Jsem si vědom toho, že na mou diplomovou práci se plně vztahuje zákon č. 121/2000 Sb., o právu autorském, zejména § 60 – školní dílo.
Beru na vědomí, že Technická univerzita v Liberci nezasahuje do mých autorských práv užitím mé diplomové práce pro vnitřní potřebu Technické univerzity v Liberci.
Užiji-li diplomovou práci nebo poskytnu-li licenci k jejímu využití, jsem si vědom povinnosti informovat o této skutečnosti Technickou univerzitu v Liberci; v tomto případě má Technická univerzita v Liberci právo ode mne požadovat úhradu nákladů, které vynaložila na vytvoření díla, až do jejich skutečné výše.
Současně čestně prohlašuji, že text elektronické podoby práce vložený do IS STAG se shoduje s textem tištěné podoby práce.
Beru na vědomí, že má diplomová práce bude zveřejněna Technickou univerzitou v Liberci v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů.
Jsem si vědom následků, které podle zákona o vysokých školách mohou vyplývat z porušení tohoto prohlášení.

6. 4. 2026 Bc. Kevin Daněk

ARCHITEKTURA SYSTÉMU POHLEDOVÉ INSPEKCE KVALITY

ABSTRAKT

Tato zpráva popisuje třídu `tulthesis` pro sazbu absolventských prací Technické univerzity v Liberci pomocí typografického systému LaTeX.

Klíčová slova: LaTeX, třída, TUL

ABSTRACT

This report describes the `tulthesis` package for Technical university of Liberec thesis typesetting using the LaTeX typographic system.

Keywords: LaTeX, class, TUL

PODĚKOVÁNÍ

V první řadě bych chtěl poděkovat vedoucímu mé práce, panu Ing. Igorovi Kopetschkovi, který mé bláznivé kvalifikační práce zastřešuje po celou dobu mého studia. Jeho lidský a pragmatický přístup mi dával naději, že nemusím hrát v absurdnímu dramatu akademické aristokracie - a že je nejenom možné, ba dokononce žádoucí, umět podat odborné téma lidsky a srozumitelně. Mrzí mě, že kdybych se rozhodl pokračovat do vyšších pater akademického studia, nebude mě moci doprovázet. Možná je to ale signálem, že je čas uvolnit místo pro ostatní, a věřte, že pokud vložíte v pana Koptschkeho důvěru, rozhodně nebudete strádat.
Dál bych chtěl poděkovat i zbytku akademické obce, se kterou jsem měl co dočinění. I když se s některými jejími členy, ozvláště pak z nadřízených pozic, neshodnu ani na počasí, tak díky každému jsem získal novou zkušenost. Někdy byla pozitivní, a někdy zase i negativní - to ale k životu patří.
A na závěr musím poděkovat svojí drahé polovičce, Simče Daňkové, která mě podporovala po celou dobu psaní této práce. Na začátku této práce byla mou přítelkyní, a nyní, na jejím konci, je již mojí manželkou. Díky ní si budu tuto práci pamatovat nejenom jako svůj *magnum opus*, ale jako životní milník, kterým se změnil skoro každý aspekt mého života. Pokud Vás zajímají regionální dějiny a příběhy z časů minulých, doporučuji si přečíst její bakalářskou práci, kterou tvoří souběžně s tou mojí, a zabývá se tématem sebevražd po druhé světové válce na Liberecku a obzvláště v Jablonci nad Nisou a jeho blízkém okolí.

OBSAH

Obsah 6
Seznam zkratek 8

1 Úvod **9**

2 Softwarové inženýrství **10**
2.1 Technický dluh 11
2.1.1 Vznik technického dluhu 12
2.1.2 Nešváry 12
2.1.3 Detekce technického dluhu 14
2.1.4 Řešení technického dluhu 15
2.2 Clean Code 16
2.2.1 Význam a záměr identifikátorů 16
2.2.2 Pravdivost a jednoznačnost identifikátorů 17
2.2.3 Sémantické odlišení identifikátorů 17
2.2.4 Hledatelnost a verbální komunikace 18
2.2.5 Očištění od technického šumu 18
2.2.6 Vlastnosti funkcí a metod 18
2.2.7 Délka a hloubka zanoření 18
2.2.8 Jednotná úroveň abstrakce 19
2.2.9 Pravidlo sestupného čtení 19
2.2.10 Rozhraní a počet argumentů 19
2.2.11 Seskupování argumentů a příznaky 20
2.3 Návrhové vzory 20
2.3.1 Specifikace návrhových vzorů 20
2.3.2 Popis vybraných návrhových vzorů 23
2.3.3 Kompozitní vzory 27
2.4 Architektonické vzory 28
2.5 Integrační vzory 28
2.6 title 28

3 Udržitelnost produkčního kódu **29**
3.1 Testování 29
3.2 Verzování 29
3.3 Postupná integrace 29
3.4 Postupné nasazení 29

3.5 Monitoring 29
3.5.1 Logy 30
3.5.2 Metriky 30

4 Audit architektury systému **31**
4.1 Modelování softwarové architektury 32
4.1.1 Úroveň systémového kontextu 32
4.1.2 Úroveň kontejnerů 33
4.1.3 Úroveň komponent 34
4.2 Modelování doménových procesů 34
4.3 Modelování procesu nasazení 34
4.4 Vyhodnocení rizik 34

5 Architektura nového řešení **37**
5.1 Postupné nasazení 37

6 Závěr **38**

Dodatek A Adaptovaná hodnotící škála vážnosti selhání **41**

Dodatek B Adaptovaná hodnotící škála výskytu selhání **42**

Dodatek C Adaptovaná hodnotící škála detekce selhání **43**

SEZNAM ZKRATEK

CD Continuous Deployment (Postupné nasazení)

CI Continuous Integration (Postupná integrace)

MES Manufacturing Execution System

IDE Integrated Development Environment (Integrované vývojové prostředí)

1 ÚVOD

Každý, jakkoliv velký, systém musí někdo navrhnout. S rostoucí složitostí systému zároveň roste i složitost jeho správného návrhu. Ve své předchozí práci jsem se věnoval rapidnímu vývoji algoritmů strojového vidění, jehož výsledkem byla nová funkční komponenta pro trénování a inferenci neuronových sítí a jiných algoritmů strojového vidění. (Daněk, 2024)

Již tehdy byla samotná implementace omezena návrhem existujícího systému. Upřednostňování rychlého dodání nových funkcionalit na úkor průběžné údržby kódu vyústilo v enormní množství technologického dluhu, který se za sedm let ve zdrojovém kódu nastřádal. Tato práce navazuje na mou předchozí práci a dívá se na ní ze širší perspektivy. Cílem je odklonit se od vývoje nových funkcí a obrátit pozornost právě k zanedbané architektuře celého řešení. Rozsah analýzy jsem stanovil nad rámec softwarového návrhu a věnuji se rovněž architektuře infrastruktury nasazeného řešení.

2 SOFTWAROVÉ INŽENÝRSTVÍ

Programátor vybavený klávesnicí má téměř neomezené možnosti realizace svých nápadů, podobně jako malíř se štětcem. Je však pozoruhodné, že na rozdíl od nekontrolovaného tvůrčího procesu malíři často své možnosti úmyslně limitují. Právě dodržením určitých kompozičních pravidel totiž docílí toho, že obraz není pouze chaotickou směsí čar a barev, ale srozumitelnou myšlenkou, kterou divák dokáže instinktivně rozpoznat.

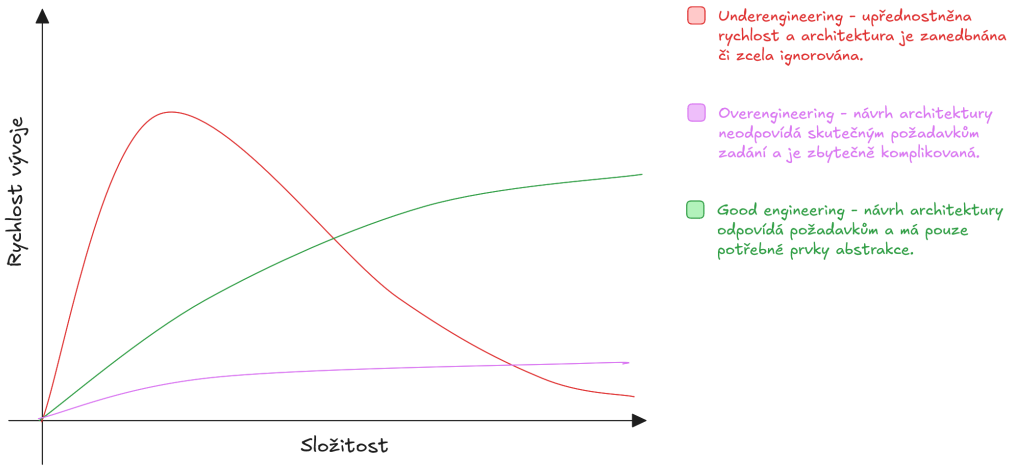
Principy vývoje softwaru se od této úvahy příliš neliší. Pokud se při vývoji zavážeme k dodržování stanovených pravidel a konvencí, umožníme ostatním vývojářům snadněji pochopit záměr našeho kódu. Této oblasti se právě věnuje **softwarové inženýrství**, které se zabývá návrhem kódu, který je udržitelný, čitelný, spravovatelný a testovatelný (Michigan Technological University, 2025). Z předvídatelného a čitelného kódu profituje i samotný autor, který se tak při návratu k vlastní práci po delším časovém odstupu nemusí věnovat náročnému dekódování původního záměru.

Softwarové inženýrství je velmi rozsáhlý obor a není v možnostech této práce jej pojmout v jeho úplnosti. Teoretický rámec se proto omezuje na vybraná témata, která jsou klíčová pro pochopení praktické části. Kapitolu otevírá pojednání o technickém dluhu, jeho příčinách a metodách jeho eliminace. Na tento koncept navazuje pasáž věnovaná principům udržitelného a testovatelného kódu, které tvoří nezbytný základ pro následné návrhové vzory. Ty využívají zmíněné principy a transformují je do osvědčených řešení běžných architektonických problémů.

Další část práce se věnuje metodice efektivního testování a principům kontinuální integrace a doručování (CI/CD). Teoretický blok uzavírá kapitola o monitoringu nasazeného řešení, který poskytuje nezbytný vhled do chování systému v produkčním prostředí, kde vývojář zpravidla nemá k dispozici stejné diagnostické nástroje jako v lokálním vývojovém prostředí.

2.1 TECHNICKÝ DLUH

Při vývoji software musí být učiněna určitá rozhodnutí o jeho návrhu. Ta jsou ovlivněna mnoha faktory - od hloubky znalosti domény a specifik zadání až po technologická omezení a časový tlak. Výsledná architektura je pak nevyhnutelně kompromisem mezi okamžitou rychlostí dodání a dlouhodobou udržitelností systému. Pod tlakem termínů musí vývojář neustále vyvažovat míru investice do čitelnosti, abstrakce a budoucí rozšířitelnosti kódu na úkor času, který má aktuálně k dispozici.



Obrázek 2.1: Rychlost vývoje v závislosti na složitosti návrhu

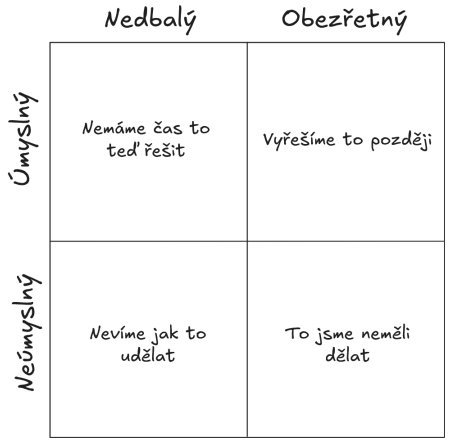
Pokud je zvolena cesta rychlejšího dodání namísto systematického návrhu, vzniká kód, který je tvořen ad hoc a postrádá ucelenou vizi. Absence celkového nadhledu pak způsobuje, že se systém s každou další funkcí stává nepřehlednějším a jeho údržba vyžaduje stále větší úsilí. Tento jev nazývám technickým dluhem. Pokud se technický dluh nechá střádat, postupně transformuje triviální změny na netriviální.

Trivialní změnou myslím takovou změnu, kdy není potřeba zasahovat do řídících struktur a logiky existujícího systému. Naopak netriviální změny vyžadují hlubší pozorumnění a studium existujícího kódu a jsou tím pádem nejen časově náročnější, ale i více zatíženy možnostmi pro vznik chyby. Příkladem triviální změny je přidání záznamu do slovníku dostupných algoritmů, nebo přidání nové obslužné metody pro údalost. Naopak příklad netriviální změny je aktualizace výrazů *switch* nebo *if* pro přidání nového algoritmu. V tento okamžik je totiž potřeba hlídat, že jsou tyto řídící struktury aktualizovány na všech relevantních místech.

Technický dluh se nazývá dluhem právě proto, že vykazuje stejné charakteristiky jako finanční závazek. Výměnou za rychlejší uvedení produktu na trh si vývojový tým od systému obrazně vypůjčuje čas, a to na úkor technické preciznosti. Stejně jako u finančních závazků, i zde je nutné počítat s úroky v podobě degradace kvality kódu.

2.1.1 Vznik technického dluhu

Vznik technického dluhu popisuje ve svých knihách a článcích Martin Fowler, který rozděluje příčiny vzniku technického dluhu do čtyř kategorií. Tyto kategorie leží na kvadrantu, který je založen na dvou osách: ose obezřetnosti a ose úmyslnosti. (Fowler, 2009)



Obrázek 2.2: Kvadranty technického dluhu (Team Asana, 2025)

Úmyslný dluh vzniká vědomým rozhodnutím upřednostnit rychlost dodání před správným návrhem. Naproti tomu neúmyslný dluh bývá důsledkem nezkušenosti nebo nedostatku znalostí v době tvorby architektury. Osa obezřetnosti pak odráží schopnost týmu zhodnotit rizika při zavádění technického dluhu: zatímco obezřetný dluh je přijat s plným vědomím budoucích nákladů na jeho splacení, nedbalý dluh vzniká s nedostatkem informací či zkušeností (Fowler, 2009).

2.1.2 Nešváry

Pokud je technický dluh podle mé dřívější definice jev způsobující celkovou nepřehlednost a neudržitelnost systému, vyvstává rozumná otázka - jak konkrétně technický dluh vypadá? Technický dluh je tvořen hromaděním kusů kódu, které degradují kvalitu softwarového řešení. Tyto kusy se nazývají *code smells*, a v této práci je nazývám **nešváry**.

Před samotným výčtem důležitých nešvárů je vhodné se pozastavit nad jejich tříděním a podívat se, jakým způsobem je odborníci klasifikují. Pro účely této práce je důležité zmínit dvě klasifikace, které na problém nahlíží zcela odlišně a přivádí do problematiky důležité myšlenky - Mäntylovu klasifikovaci a Perezovu klasifikaci. Mäntylä nahlíží na nešváry jako na skupiny příbuzných symptomů. Dívá se tedy na nešváry optikou jejich dopadu na kód samotný. Jednotlivé kategorie definoval na základě společného negativního dopadu na kvalitu kódu (Mäntylä, 2010; Shvets, 2018a):

- **Nabobtnalci** - tedy kód, který se rozrostl do takových proporcí, že s ním nelze nadále efektivně pracovat;
- **Zneužívači** - tedy kód, který špatně využívá možnosti objektového návrhu;
- **Zabraňovači** - tedy kód, který ztěžuje změny či další vývoj;
- **Zbytečníci** - tedy zbytečné části kódu, které ztěžují jeho pochopení;
- a **Vazbiči** - tedy případy, kdy je kód nesprávně provázaný a dochází ke špatnému rozdělení odpovědnosti.

Alternativní klasifikaci nabízí právě Perez, který netřídí nešváry do skupin podle společných vlastností, ale přišel s myšlenkou, že tyto nešváry tvoří de-facto monoid s operací skládání. Perez totiž popisuje některé nešváry jako nízkoúrovňové a atomické, tudíž jsou ireducibilní a tvoří základní kámeny pro skládání složitějších nešvárů, které nazývá vysokoúrovňovými (Perez, 2011). Tento pohled na nešváry závádí do problematiky důležitou myšlenku - že nešváry nejsou disjunktními problémy, ale že i v případě Mäntylovy klasifikace existuje míra překrytí.

Právě Pérezova myšlenka hierarchického skládání nešvarů vysvětluje, proč je v praxi obtížné stanovit mezi Mäntylovými kategoriemi ostré hranice. Atomické nešváry, jako je například *duplicitní kód*, totiž často fungují jako katalyzátory pro vznik složitějších, vysokoúrovňových problémů. Pokud se identická logika vyskytuje na mnoha místech systému, nevyhnutelně to vede ke *shotgun surgery*, kdy i triviální změna požadavku vynucuje koordinovanou netriviální změnu napříč rozptýlenými moduly.

Kromě této přímé posloupnosti, kdy jeden nešvar vyvolává druhý, však v kódu existují i vztahy, které připomínají spíše korelaci než jasnou kauzalitu. Typickým příkladem je právě *velká třída* (Large Class). Jakožto zástupce *nabobtnanců* má symptom mnoha řádků kódu, ale v těchto řádcích se mohou ukrývat další nešváry, které nemají s velkou třídou kromě kolokace nic společného. Perez by hledal přímou souvislost mezi těmito jevy, tudíž že všechny dohromady skládají velkou třídu. Mäntyla ale dovoluje uvažovat bezpříčinné překrytí kategorií a soustředit se na kvantitativní výčet.

Volba mezi těmito dvěma klasifikacemi pak v praxi závisí především na cíli samotné analýzy. Zatímco Pérezův přístup je nepostradatelný při snaze pochopit hloubkovou strukturu dluhu a identifikovat jeho primární příčiny, Mäntylova klasifikace nabízí přímočařejší cestu ke kvantitativnímu výčtu symptomů. Pro vývojáře je v danou chvíli důležité se rozhodnout, zda se zaměří na sledování kauzálních řetězců a bude se snažit eliminovat dluh u jeho kořene, nebo zda využije horizontální překryv kategorií k rychlé identifikaci nejvíce degradovaných modulů.

V následující tabulce 2.1 jsem vybral nešváry, které považuji za stěžejní pro praktickou část této práce:

Tabulka 2.1: Výběr nešvarů, jejich symptonů a příčin

Název	**Symptomy**	**Příčiny**	**Možné řešení**
Dlouhá metoda	Metody nad 20 řádků.	Nedodržení principu jedné odpovědnosti	Rozdělení na více metod
Velká třída	Třída, která dělá více věcí zároveň	Nedodržení principu jedné odpovědnosti.	Rozdělení na více tříd
Divergentní změna	Mnoho úprav v jednom souboru.	Kopírování kódu.	Vytvoření třídy
Shotgun Surgery	Jedna úprava v mnoha souborech.	Jedna odpovědnost v mnoha třídách.	Vytvoření třídy
Duplicitní kód	Stejná logika na více místech.	Kopírování kódu.	Vytvoření metody
Mrtvý kód	Nevyužité metody/proměnné.	Strach ze smazání kódu.	Odstranění kódu
Switch Statements	Rozsáhlé switch/if struktury.	Neobratné využití OOP.	Polymorfismus
Magické literály	Čísla a řetězce v řídících strukturách	Chybějící abstrakce, snaha o rychlé řešení	Nahrazení symbolickou konstantou

2.1.3 Detekce technického dluhu

Každý nešvár přispívá k celkovému technickému dluhu. Nešváry jsou příčinou, technický dluh stavem a netrivialita změn následkem. V důsledku se tak technický dluh v projektu projevuje jako překážka v životním cyklu softwaru, kterou je nutné nejprve identifikovat, zmapovat a zpětně dotrasovat k nešvárům.

Prvotním indikátorem přítomnosti technického dluhu je obvykle dynamika samotného vývoje. Pokud se rychlost vývoje snižuje a čas potřebný k implementaci nových funkcionalit či opravě zdánlivě prostých chyb roste, bývá jednou z příčin právě nutnost vývojářů neustále překonávat bariéry v podobě zanedbaného či nepřehledného kódu. Právě samotní vývojáři nabízí sice subjektivní, avšak kriticky důležité hodnocení. Ti jsou totiž schopni přesně identifikovat konkrétní úseky systému, jež je při práci nejvíce zpomalují.

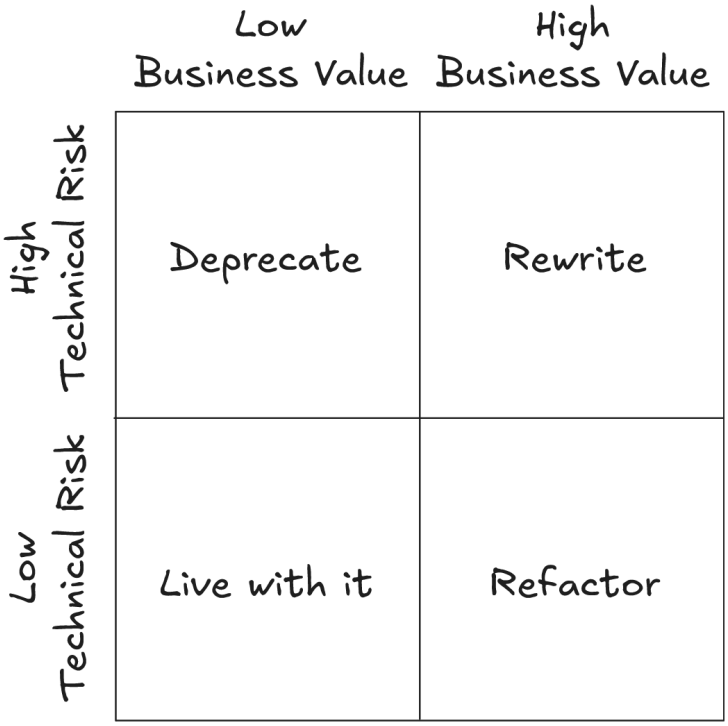
Objektivnější pohled nabízejí nástroje pro statickou analýzu kódu, které dokáží automatizovaně detekovat duplicity nebo antipatterny. Významnou metriku představuje rovněž code churn – ukazatel četnosti změn v konkrétních částech kódu. Vysoká míra netriviálních změn v určitém modulu často signalizuje jeho nevhodný návrh, nestabilitu nebo přílišnou provázanost s ostatními částmi systému, což jej činí náchylným k chybám a náročným na údržbu (Team Asana, 2025).

2.1.4 Řešení technického dluhu

Volba strategie pro řešení technického dluhu vyžaduje analýzu rozsahu, souvisejících rizik a dostupných zdrojů. Pro hodnocení lze využít standardních postupů jako analýza SWOT, výpočet poměru nákladů na vývoj a na opravu (*Technical Debt Ratio*) a nebo pokročilejších analýz pro hodnocení rizik, které si lze přizpůsobit pro účely hodnocení technického dluhu. (SonarSource, 2025)

Základní metodu pro vyhodnocení strategie splácení technického dluhu představuje rozhodovací matice, která dělí systémy do čtyř kategorií na základě dvou os: přínos pro podnik a technické riziko. Právě přínos pro podnik určuje, zda má smysl do daného systému investovat zdroje – pokud je vysoký, nemůžeme si dovolit nechat dluh nekontrolovaně hromadit a musíme zvolit cestu nápravy.

O konkrétním postupu pak rozhoduje osa technického rizika, která posuzuje pravděpodobnost selhání a závažnost jeho dopadů. V situaci, kdy je kód relativně stabilní nebo jsou následky případné chyby přijatelné, se jako ideální cesta jeví postupná refaktorizace. Ta umožňuje kontinuální splácení dluhu za plného provozu, čímž minimalizuje rizika regrese. Naopak v extrémním případě, kdy je systém pro podnik kritický, ale zároveň vykazuje neúnosnou míru rizika, se kompletní přepis stává nevyhnutelným. Přepis tak v tomto kontextu nepředstavuje selhání údržby, ale strategické rozhodnutí o vybudování nového, udržitelného základu.



Obrázek 2.3: Kvadranty řešení technického dluhu (ForrestKnight, 2025)

2.2 CLEAN CODE

Každý pokus o odstranění technického dluhu v praxi dříve či později naráží na nutnost provést v kódu netriviální změnu. Jak bylo definováno v kapitole 2.1, taková operace vyžaduje nejen hluboké pochopení stávající logiky a řídicích struktur, ale také značnou dávku kognitivního úsilí. Míra této zátěže však není dána pouze složitostí samotného problému, ale především kvalitou a stylem, jakým byl kód napsán.

Pro systematické snižování této kognitivní zátěže se v softwarovém inženýrství ustálil koncept *Clean Code*. Jedná se o soubor principů a doporučení, jejichž primárním cílem je zjednodušit čtení a údržbu zdrojového kódu. Aplikace těchto zásad v praxi přímo koreluje s kognitivní zátěží vývojáře: čím je kód „čistší“, tím méně mentálních kapacit musí věnovat jeho dešifrování a tím více prostoru zbývá na samotnou refaktorizaci a tvorbu nových funkcionalit (Martin; Coplien, 2009).

Clean Code není pouhým souborem pravidel, ale především způsobem uvažování o kódu jako o komunikačním prostředku. Jeho cílem je sjednotit sdělení záměru autora a zjednodušit budoucí údržbu či rozšiřování systému. Jak uvádí Robert Martin, jednotlivé zásady nelze vnímat jako absolutní dogmata; jejich konkrétní aplikace je vždy výsledkem odborné diskuse v rámci týmu (Martin; Coplien, 2009).

2.2.1 Význam a záměr identifikátorů

Pokud rozvineme myšlenku, že kód je komunikačním prostředkem, pak nejdůležitějším nástrojem této výměny informací je právě volba jmen. Nevhodné pojmenování je totiž nejčastější formou nízkoúrovňového technického dluhu – je levné jej vytvořit, ale jeho kumulativní úrok v podobě kognitivního úsilí při každém dalším čtení je enormní.

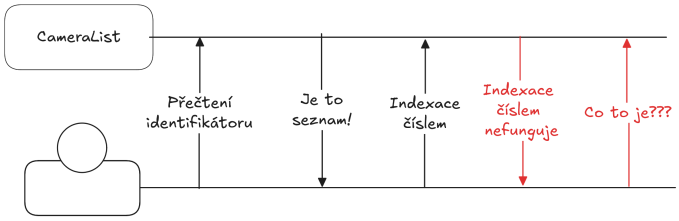
Základním stavebním kamenem čitelnosti je požadavek, aby každý název nesl jasný význam a záměr. Identifikátor by měl bez nutnosti nahlížet do implementace odpovídat na otázky:

- Proč tato entita existuje?
- Co tato entita dělá?
- Jakým způsobem se entita používá?

Zatímco jednopísmenný či zkratkovitý název proměnné je pro čtenáře pouze šumem, název jako `delkaPreponyCm` v sobě integruje informaci o fyzikálním významu i jednotkách, čímž okamžitě snižuje mentální náročnost pochopení daného výpočtu (Martin; Coplien, 2009).

2.2.2 Pravdivost a jednoznačnost identifikátorů

Kromě významu a záměru identifikátoru musíme dbát také na jeho integritu – název zkrátka nesmí lhát. Pokud kód obsahuje dezinformace, dochází k rozporu mezi účelem a skutečnou implementací. Označení proměnné jako `seznamUzivatelu`, přestože se ve skutečnosti jedná o slovník, vnáší do kódu zavádějící intuici. Vývojář na základě názvu podvědomě očekává vlastnosti seznamu, jako je garantované pořadí nebo indexace přirozenými čísly. Jakmile je tento předpoklad při ladění programu vyvrácen, dochází k okamžité ztrátě důvěry, neboť každá další konstrukce může také lhát.

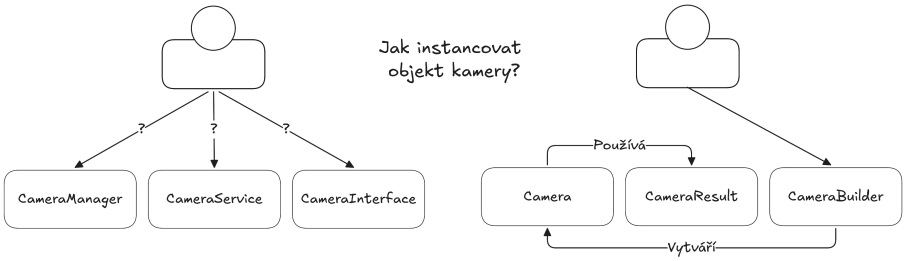


Obrázek 2.4: Nepravdivé názvy vytvářejí překážky ve vývoji

Stejně rizikové jsou i příliš podobné názvy. V éře moderních IDE, která využívají našeptávání kódu či AI asistenty, může minimální rozdíl mezi dvěma identifikátory vést k těžko odhalitelným chybám. Tento problém se navíc stupňuje s rostoucí délkou identifikátorů. Snaha o extrémní popisnost může paradoxně vést k tomu, že se podstatná informace ztratí v záplavě technického šumu (Martin; Coplien, 2009).

2.2.3 Sémantické odlišení identifikátorů

V návaznosti na jednoznačnost identifikátorů je nutné zohlednit také jejich sémantické odlišení. Pokud jsou identifikátory jednoznačné, ale odlišené jsou pouze pomocí sufixů (např. `data1`, `data2`) nebo vágních afixů jako `manager` či `info`, je vývojář odkázán na vlastní interpretaci vágního názvu nebo na studium implementace. Taková jména vznikají většinou v momentě, kdy vývojář rezignuje na hledání sémantického rozdílu mezi dvěma entitami. Pokud v systému existují dvě různé proměnné, musí existovat i sémantický důvod pro jejich odlišení, který by se měl promítnout do jejich názvu.



Obrázek 2.5: Semantické identifikátory implikující vazby mezi objekty

2.2.4 Hledatelnost a verbální komunikace

Kód není určen pouze pro kompilaci, ale i pro komunikaci a diskuzi uvnitř týmu. Identifikátory by proto měly být snadno vyslovitelné, aby se předešlo komickému, ale neefektivnímu hláskování nesrozumitelných zkratek. S tím souvisí i vyhledatelnost; unikátní a popisný název umožňuje vývojáři okamžitou orientaci v projektu. V tomto kontextu je kritické nahrazovat tzv. „magické konstanty“ pojmenovanými konstantami, které do řídicích struktur vnášejí potřebnou úroveň abstrakce (Martin; Coplien, 2009).

2.2.5 Očištění od technického šumu

Clean Code rovněž varuje před kódováním technických informací přímo do názvů. V éře silné statické analýzy je redundantní uvádět v názvu datový typ nebo prefixovat rozhraní písmenem `I`. Případná kolize názvu rozhraní a třídy (např. `Camera`) by neměla být řešena technickým prefixem `ICamera`, ale raději sémantickým upřesněním konkrétní role – například `CameraService` nebo `PhysicalCamera`. Takový přístup vkládá do názvu informaci o principu fungování, nikoliv pouze o syntaktické kategorii (Martin; Coplien, 2009).

2.2.6 Vlastnosti funkcí a metod

Zatímco vhodná jména tvoří slovní zásobu, funkce a procedury představují jeho gramatiku. Jsou první skutečnou úrovní abstrakce a základním nástrojem pro dekompozici složité logiky. *Clean Code* proto definuje tři pilíře pro jejich návrh:

- funkce by měla být krátká,
- měla by vykonávat právě jednu činnost
- a její působení musí být omezeno na vlastní kontext bez nechtěných *vedlejších účinků* (Martin; Coplien, 2009)

2.2.7 Délka a hloubka zanoření

Požadavek na minimalizaci délky funkce není samoúčelný. Jedná se především o obranu před kognitivním přetížením. Příliš dlouhá metoda s vysokou mírou zanoření v řídicích strukturách (cykly, podmínky) nutí vývojáře udržet v hlavě příliš mnoho stavových proměnných najednou. Jak trefně poznamenal Linus Torvalds: *„if you need more than 3 levels of indentation, you're screwed anyway, and should fix your program.“* (Torvalds, 2007). Každá úroveň zanoření navíc představuje ideální místo pro extrakci do samostatné, pojmenované metody s jasnou zodpovědností.

2.2.8 Jednotná úroveň abstrakce

Princip jediné činnosti vyžaduje, aby si funkce zachovala konzistentní úroveň abstrakce. Funkce by měla buď vykonávat příkaz, nebo odpovídat na dotaz, ale nikdy obojí současně (princip *Command Query Separation*). Strategická logika by neměla být mísena s nízkoúrovňovými detaily, jako je formátování řetězců nebo výpočty. Tento hierarchický přístup lze přirovnat k fungování firmy: ředitel (vysokoúrovňová funkce) vydává strategické pokyny, zatímco operativní detaily deleguje na vedoucí skladů, kteří řídí konkrétní naskladnění zboží (Martin; Coplien, 2009).

2.2.9 Pravidlo sestupného čtení

Pro udržení této hierarchie abstrakce je doporučeno dodržovat pravidlo sestupného čtení (*The Stepdown Rule*). Program by měl být čitelný jako plynulé vyprávění, kde každý odstavec rozvíjí detaily předchozího. V kontextu objektového návrhu to znamená, že implementační detaily následují až po metodách, které je volají (Martin; Coplien, 2009):

- K vyřízení objednávky je třeba zabalit zboží, odečíst jej ze skladu a vytisknout štítek.
- K zabalení zboží je třeba položku vyhledat a zvolit vhodný obal.
- K odečtení ze skladu je třeba identifikovat pozici zboží v registru.

2.2.10 Rozhraní a počet argumentů

Komplexitu funkce však neurčuje pouze její tělo, ale také šířka jejího rozhraní. Ideální funkce je niladická (bez argumentů). Monadické a dyadické funkce jsou běžné a přijatelné, ale s každým dalším parametrem roste obtížnost pochopení záměru a dramaticky se komplikuje testování.

Uvažujeme-li funkci se čtyřmi parametry typu *boolean*, vzniká celkem 2^4 možných kombinací vstupních stavů, které je v ideálním případě nutné validovat. Obecně je pak počet všech teoreticky možných testovacích případů roven mohutnosti kartézského součinu množin přípustných hodnot všech jednotlivých argumentů. Pro k argumentů stejného výčtového typu o n prvcích je tento počet dán vztahem n^k. U spojitých proměnných je situace ještě kritičtější, neboť vyžaduje pečlivý výběr reprezentativních zástupců z tzv. tříd ekvivalence, aby bylo možné pokrýt hraniční stavy bez nekonečného nárůstu testovacích scénářů (Martin; Coplien, 2009).

Vysoký počet argumentů navíc smazává přirozené pořadí parametrů, což zvyšuje riziko jejich záměny při volání – výjimkou jsou pouze případy kdy mezi parametry již existuje přirozené uspořádání, jako jsou například souřadnice v kartézské soustavě (x, y). (Martin; Coplien, 2009)

2.2.11 Seskupování argumentů a příznaky

Pokud funkce vyžaduje velký počet parametrů, je to jasný signál, že některé z nich tvoří samostatný koncept a měly by být seskupeny do logické datové struktury či objektu. *Clean Code* rovněž důrazně varuje před používáním příznakových argumentů (*flag arguments*). Pokud funkce přijímá *boolean*, který větví její tok, z definice vykonává více než jednu věc a měla by být rozdělena do samostatných, jasně pojmenovaných funkcí. (Martin; Coplien, 2009).

2.3 NÁVRHOVÉ VZORY

Zatímco *Clean Code* se soustředí na disciplínu v nejnižší úrovni abstrakce – tedy na to, jak psát čitelný a srozumitelný kód v rámci funkcí a tříd. Návrhové vzory posouvají tuto snahu o řád výše. Představují totiž konceptuální řešení častých problémů v návrhu softwarové architektury. Na rozdíl od algoritmu, který definuje striktní sekvenci kroků k řešení problému, jsou návrhové vzory spíše obecnými šablonami či strategiemi. Vývojář tyto strategie adaptuje v souladu se specifiky programovacího jazyka a potřebami daného systému (Shvets, 2018b). Vzory tak přenášejí žádoucí vlastnosti *Clean Code* do vyšších úrovní abstrakce, kde se formují rozsáhlejší architektonické celky. Skrze jejich použití definujeme nejen záměr a myšlenku za existencí jednotlivých komponent, ale i jejich očekávané chování a vzájemné interakce. Rozpoznání vzoru v kódu tak vývojáři okamžitě odhaluje sémantiku celé komponenty bez nutnosti studovat detailní implementaci.

2.3.1 Specifikace návrhových vzorů

Popis samotných návrhových vzorů se v praxi pohybuje na různých úrovních formality. Nejčastěji se setkáváme s neformálními popisy, které kombinují text a grafické notace, jako jsou blokové diagramy nebo UML diagramy (Shvets, 2018b). Tyto vizualizace jsou sice dobrou reprezentací návrhového vzoru, ale neposkytují dostatečně pevný základ pro kontrolu vlastností navrženého systému. Pokud tedy chceme analyzovat kód a hledat místa pro využití návrhových vzorů, vzniká otázka, jak je alespoň kvaziformálně definovat a porovnávat.

Formalismy nejsou přitom softwarovému oboru cizí. V oblasti návrhu a teorie databází slouží k popisu manipulace s daty relační algebra a k definici logických podmínek relační kalkul. I v objektovém návrhu vznikaly snahy o zavedení formálního aparátu, který by dokázal modelovat vztahy a komunikaci mezi objekty při zachování klíčových vlastností OOP, jako je zapouzdření nebo schopnost kompozice. Cílem těchto pokusů bylo tedy vytvořit tzv. objektový kalkul.

Nierstraszova notace

Významným příspěvkem k formalizaci objektového návrhu byla práce Oscara Nierstrasze, který pro tento účel propojil lambda kalkul s procesním kalkulem. Volba těchto dvou disciplín byla strategická: lambda kalkul použil k definici vnitřních stavů a chování objektů, zatímco procesní kalkul použil pro modelování vzájemné komunikace objektů. Nierstrasz tak využil silné stránky dvou etablovaných disciplín a spojil je do jednoho, zdánlivě stejně silného, aparátu, který nazývám *Nierstraszova notace* (Nierstrasz, 1992).

Ve své notaci Nierstrasz představil koncept agentů, které reprezentují objekty. Agenti jsou uspořádanou n-ticí definovánou třemi množinami: jména, vstupy a výstupy. Mezi těmito agenty následně vytvořil řadu operátorů, například pro modelování rekurze, kompozice, aplikace funkcí a dalších prvků objektového i funkcionálního programování, které jsou ukázány v rovnici 2.1 (Nierstrasz, 1992).

$$a ::= a\&a \mid n := a \mid a|a \mid x \rightarrow a \mid v \wedge a \mid a@v \tag{2.1}$$

Konkrétní práci s Nierstraszovou notací lze ukázat na definici prázdného zásobníku: $empty := (push, x?) \rightarrow stack@(x, empty)$. Tento zápis popisuje objekt, který disponuje operací *push* s lokálním argumentem x. V momentě, kdy objekt přijme zprávu s touto signaturou, transformuje se na instanci *stack*, které jsou předány parametry v podobě přijatého argumentu a reference na původní objekt *empty*. Logika této transformace vychází z definice zásobníku v rovnicích 2.2 a 2.3, kde je *stack* koncipován jako entita přijímající parametry *top* (aktuální hodnota x) a *rest* (předchozí stav zásobníku) (Nierstrasz, 1992).

$$stack := (top?, rest?) \rightarrow (pop, top) \wedge rest \tag{2.2}$$

$$stack := (top?, rest?) \rightarrow (push, x?) \rightarrow stack@\,(x, stack@(top, rest)) \tag{2.3}$$

Z rovnice 2.2 je vidět, že operace *pop* vrací hodnotu *top* a zároveň obnovuje stav *rest*, čímž je realizována struktura LIFO. Naopak rovnice 2.3 ukazuje rekurzivní povahu operace *push*, která vytváří novou instanci zásobníku zapouzdřující stav té předchozí.

Ačkoliv Nierstrasz skvěle modeluje předávání zpráv, z pohledu detekce návrhových vzorů je příliš nepoužitelný. Nutí nás sledovat každou jednotlivou zprávu a transformaci agenta, abychom pochopili, že se jedná i o obyčejný zásobník. S rostoucí složitostí systému poroste i náročnost popisu rozsáhlých architektur.

Zásadním nedostatkem je z mého pohledu také absence explicitního vyjádření vazeb mezi objekty. Nierstraszova notace se soustředí primárně na sekvenci zpráv, čímž v podstatě vytváří formální verzi sekvenčního diagramu.

Mikkonenova notace

Na problém vyjádření vazeb již netrpí Mikkonenova notace, která je založena na metodě DisCo. Mikkonen se odklání od popisu komunikace pomocí zpráv a soustředí se na objekty jako na součásti širšího celku, přičemž klade důraz na jejich vzájemné vztahy. Jednotlivé třídy definuje jako množiny atributů a relací, přičemž relace mají explicitně vyjádřenou kardinalitu. Obecný zápis takové relace uvádí rovnice 2.4, kde je vztah definován jako výběr z kartézského součinu dvou tříd (Mikkonen, 1998).

$$(n) \cdot R \cdot (m) : C \times D \tag{2.4}$$

Konkrétní příklad relace ukazuje rovnice 2.5 na relaci *Attached* mezi třídami *Subject* a *Observer*. Tento zápis definuje, že k libovolnému počtu instancí třídy *Observer* může být připojen buď jeden konkrétní *Subject*, nebo žádný (vazba neexistuje). Relace tedy umožňuje, aby se jeden *Subject* registroval u více pozorovatelů, zatímco každý *Observer* může nekonečně mnoho vazeb a jednotlivé instance třídy *Subject* (Mikkonen, 1998).

$$(0..1) \cdot Attached \cdot (*) : Subject \times Observer \tag{2.5}$$

Jednotlivé relace nevznikají svévolně; ačkoliv mohou být pevně definovány již při inicializaci systému, mnohdy se ustavují dynamicky v čase prostřednictvím společných akcí (joint actions). Akce je v tomto pojetí chápána jako funkce, do níž vstupuje více objektů současně. Každá taková operace disponuje seznamem účastníků, aktivační podmínkou a definicí následných změn stavů (Mikkonen, 1998).

$$A(c : C, i) : i \neq c.x \rightarrow c.x' = i \tag{2.6}$$

Příklad takové akce uvádí rovnice 2.6. Akce A přijímá jako parametry instanci c třídy C a proměnnou i. Pokud je splněna podmínka, že se hodnota i nerovná aktuálnímu atributu x objektu c, dojde k aktualizaci tohoto atributu na hodnotu i. V podstatě se jedná o formální model setteru třídy, respektive atomické přiřazení nové hodnoty atributu (Mikkonen, 1998).

Ačkoliv je Mikkonenova notace pro popis architektury a návrhových vzorů výrazně lepším nástrojem než Nierstraszova notace, stále zůstává příliš svázána s konkrétním vnitřním stavem komponent. Mikkonen například při formalizaci vzoru *Observer* operuje s pevně definovanými atributy subjektů, jako je *Data* (Mikkonen, 1998). I když jsou samotné vazby popsány skvěle, tento přežitek vnitřního stavu v praxi komplikuje srovnání ideální struktury vzoru s reálným zdrojovým kódem. V praktickém nasazení je totiž téměř nemožné dokázat shodu formálního popisu s existujícím řešením, pokud se implementační detaily (názvy a počty atributů) byť jen minimálně odchylují od vzoru.

Teorie kategorií

Právě dříve zmíněné nedostatky obou notací mě zavedly na cestu snahy o vlastní způsob formalizace návrhových vzorů. Cílem bylo nalézt rigorózní základ, který by nebyl zkostnatělý a nabídl by dostatečnou flexibilitu pro popis struktury i dynamických vztahů. Prvotním impulzem byl pokus o syntézu Nierstraszovy a Mikkonenovy notace – snaha získat z obou modelů jejich nejsilnější stránky, podobně jako Nierstrasz kdysi sjednotil lambda kalkul s procesním kalkulem.

Brzy jsem však narazil na ty samé problémy, za které jsem Nierstraszovu notaci kritizoval. S každou další úpravou a pokusem o zpřesnění modelu jsem se ocital v situaci připomínající Newtonovu metodu hledání kořene polynomu: sice jsem se v jednotlivých iteracích přibližoval k cíli, ale za cenu neúměrného nárůstu komplexity a ztráty elegance celého popisu.

Rozhodl jsem se proto udělat krok zpět, a podívat se na již známé matematické koncepty, které by mohly sloužit jako univerzální základ pro popis návrhových vzorů. Mé prvotní úvahy směřovaly k diskrétní matematice a abstraktní algebře, konkrétně k lineárním zobrazením. Ta sice dokáží efektivně modelovat vazby, jsou však stále příliš úzce asociována s funkcemi či množinami. Potřeboval jsem jít z hlediska abstrakce výše.

V tento okamžik jsem si vzpomněl na teorii kategorií, o kterou jsem se zlehka v minulosti zajímal. Jedná se pravděpodobně o nejabstraktnější disciplínu pro popis matematických struktur a jejich vzájemných vztahů, stojící na pomezí moderní abstraktní algebry a teorie grafů. Experimentální aplikace této myšlenky mi potvrdila, že teorie kategorií nabízí nejen slušný způsob popisu samotných návrhových vzorů, ale dává mi i nástroje pro identifikaci vhodných míst pro jejich aplikaci v existujícím kódu. Rozhodl jsem se proto v této linii pokračovat; konkrétní výsledky a formální rámec mého přístupu jsou součástí následující podkapitoly.

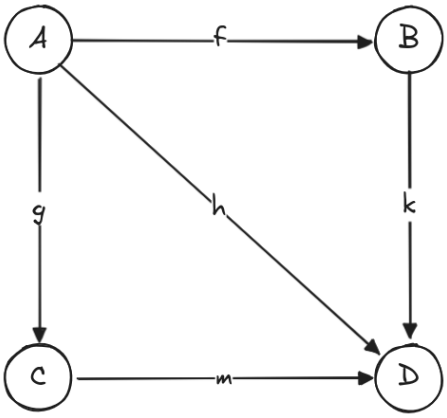
2.3.2 Popis vybraných návrhových vzorů

Jak vyplývá z předchozí kapitoly, k formálnímu popisu vybraných návrhových vzorů využiji teorii kategorií. Je však nutné zdůraznit, že tato práce není zaměřena na teorii kategorií jako takovou, ani si neklade za cíl její hloubkový teoretický rozvoj. Cílem je využít základní principy a abstrakce, které tato disciplína zavádí, a aplikovat je na modelování softwarových struktur s co největším důrazem na matematickou přesnost. Před samotným užitím teorie kategorií je třeba rozvést, co je vůbec jejím předmětem - kategorie, morfismy a funktory.

Kategorie je struktura tvořená množinou objektů a množinou morfismů. Objekty jsou reprezentací předmětů zkoumání - funkce, prostory, metriky, a v našem případě softwarové třídy a funkční jednotky. Objekty značím velkými písmeny latinky. (Ma, 2026)

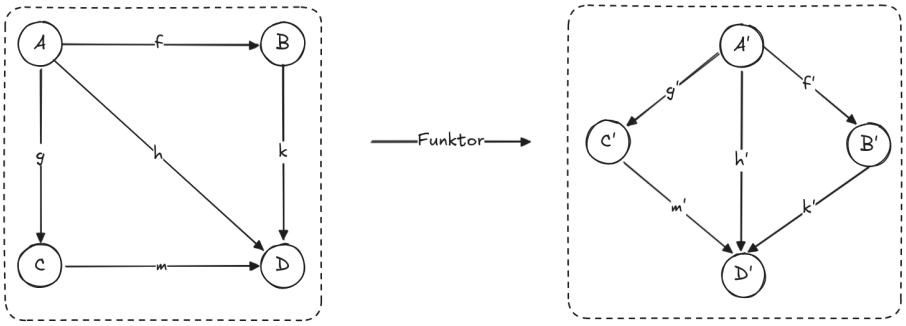
Morfismy jsou zobrazením mezi objekty. Slouží jako nositelé vazeb, transformací, operací a klidně i samotných objektů, pokud zůstanou dodržena pravidla skládání. Morfismy značím malými písmeny latinky. (Ma, 2026)

Kategorie, respektive jejich objekty a morfismy, se vizuálně zakreslují do diagramů, které strukturou připomínají orientované grafy. Diagram označujeme jako komutativní, pokud všechny morfismy se stejným definičním oborem a oborem hodnot představují jeden a ten samý morfismus. Komutativní vlastnost je důležitá, protože když není dodržena, můžeme se z bodu A dostat do bodu B, a pokaždé skončit s jiným výsledkem. Když diagram komutuje, máme zaručené, že výsledek bude vždy stejný. (Ma, 2026)



Obrázek 2.6: Komutativní diagram

Diagramy slouží k vizuální reprezentaci vnitřní struktury kategorií, přičemž tyto struktury nejsou vázány na konkrétní instance – různé kategorie mohou sdílet identické uspořádání objektů a morfismů. Tato strukturální podobnost vede ke konceptu ekvivalence kategorií a možnosti definovat přechody mezi nimi. K tomuto účelu slouží funktor – speciální typ zobrazení, který převádí jednu kategorii do druhé, aniž by narušil vztahy mezi definičními obory a obory hodnot. (Ma, 2026)



Obrázek 2.7: Funktor

Pokud návrhové vzory formalizuji jako kategorie a reprezentuji je pomocí komutativních diagramů, lze v kódu vyhledávat nejen přesné strukturání shody, ale také identifikovat částečné shody o různé vzdálenosti vůči ideálnímu vzoru či definovaným protivzorům. Pro tento účel je potřeba zavést několik relací mezi objekty kategorie:

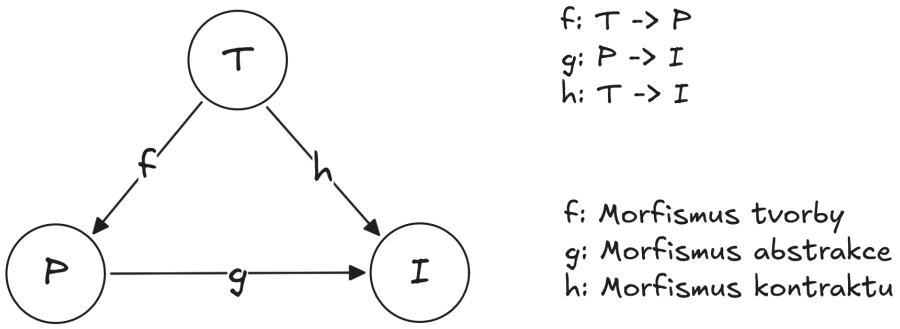
- **Sémantická ekvivalence** ($A \approx B$) – Objekty jsou na sobě strukturálně nezávislé, ale sdílejí totožný koncepční záměr. V daném doménovém kontextu jsou logicky zaměnitelné, což indikuje potenciál pro zavedení společné abstrakce (např. Factory či Interface).
- **Izomorfismus** ($A \cong B$) – Objekty vykazují naprostou shodu v rozhraní i struktuře. Tato relace představuje ideální stav čistého návrhu, kdy jsou entity plně a bezpečně zaměnitelné bez vedlejších účinků na okolní systém.
- **Kongruence** ($A \equiv B$) – Objekty sdílejí shodu v konkrétní podmnožině metod či vlastností, ale nejsou jimi plně vázány. Typicky se jedná o projev *duck typingu*; objekt sice disponuje očekávaným rozhraním, ale jeho vnitřní implementace či dodatečný balast narušují čistotu vzoru.
- **Strukturální podobnost** ($A \sim B$) – Objekty vykazují částečně shodné strukturní rysy (např. podobné grafové topologie), přestože mezi nimi neexistuje přímá vazba či společný předek.

Formální definici těchto relací upozadím ve prospěch jejich širší aplikace. Důležité rozlišení mezi těmito relacemi je, že sémantická ekvivalence spojuje shodný záměr; izomorfismus shodné objekty; kongruence shodu v rozhraní a strukturální podobnost shodu ve struktuře.

Ještě se dovolím pozastavit nad rozdílem mezi izomorfismem a kongruencí, který je podle mého názoru zásadní pro pochopení jejich odlišení. Relací izomorfismu říkám, že objekty mají rozhraní shodné do posledního detailiu. Relací kongruence zase říkám, že objekty mají průnik v rozhraní, ale objekty nabízí ještě další metody a atributy, které nejsou součástí společného rozhraní. To je nebezpečné v situaci, kdybychom chtěli tyto objekty zaměnit. Jakákoliv budoucí záměna takového objektu vyžaduje ošetření všech těchto volání.

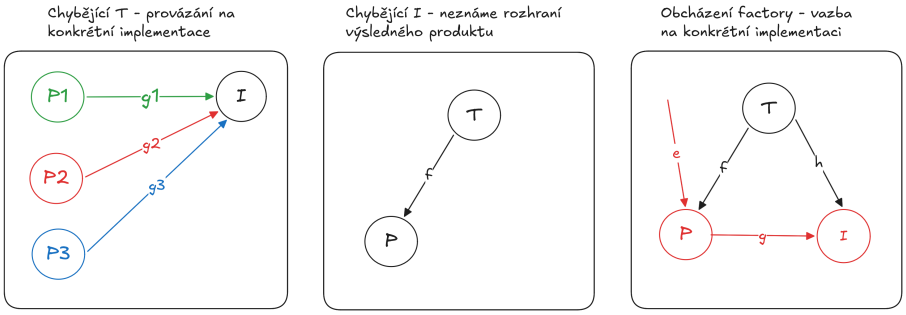
Factory

Návrhový vzor factory řeší problém přílišného provázání s konkrétní implementací třídy, i když existuje více tříd, které jsou sémanticky zaměnitelné díky stejnému rozhraní. Skrytím tvorby konkrétních objektů za abstraktní požadavek dochází k odstínění klienta od implementace, kterou může nyní měnit za běhu programu. Přidání nových typů produktů nevyžaduje modifikaci stávajícího kódu potenciálně na mnoha místech (*shotgun surgery*), ale je centralizované právě do jedné třídy.



Obrázek 2.8: Komutativní diagram návrhového vzoru: Factory

Formálně je factory kategorií o objektech *Type*, *Product*, *Interface* a o morfismech tvorby, abstrakce a kontraktu. Objekt *Type* představuje požadavek na vytvoření konkrétního typu objektu, například pomocí textového řetězce v parametru metody. Objekt *Product* představuje konkrétní implementaci třídy a objekt *Interface* tvoří společné rozhraní pro všechny produkty. Morfismus tvorby představuje vytvoření či předání třídy pomocí požadavku, morfismus abstrakce zavazuje všechny produkty, aby implementovali společné rozhraní, a morfimus kontraktu zajišťuje, že při požadavku pomocí typu bude vrácena skutečně třída se společným rozhraním.



Obrázek 2.9: Možné protivzory návrhového vzoru: Factory

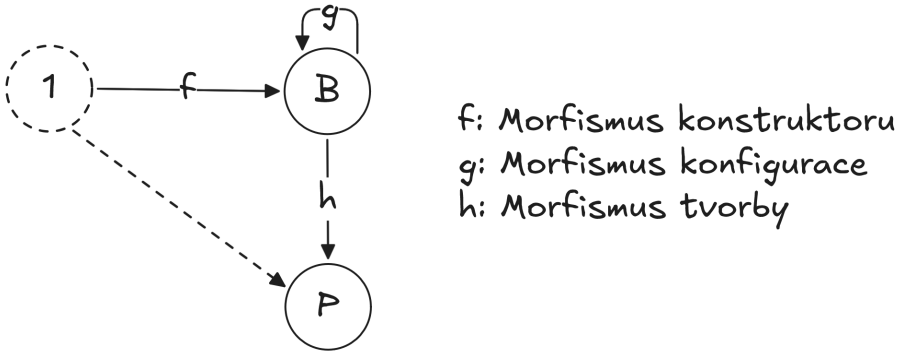
Factory je tedy vhodné využít, pokud existují třídy, které mají shodné, či dostatečně blízké rozhraní, jsou sémanticky příbuzné a jsou v systému zaměňovány podle kontextu.

$$\forall P_i, P_j \in S \wedge P_i \neq P_j : P_i \sim_{sem} P_j \implies \exists I (P_i \sqsubseteq I \wedge P_j \sqsubseteq I) \tag{2.7}$$

V takovém případě absence vzoru vede k fragmentaci logiky vytváření tříd, zvyšuje provázanost systému a při nutné změně vytváří nešvár *Shotgun Surgery*, který požadavek na přidání nového typu produktu tvoří netriviálním. Naopak u jednoúčelových tříd, kde se nepředpokládá budoucí existence alternativních implementací se shodným rozhraním, by byla aplikace tohoto vzoru kontraproduktivní.

Builder

Návrhový vzor *Builder* řeší nešváry převážně z třídy nabobtnalců a zneužívačů objektového návrhu. Dovoluje nám totiž krok za krokem sestavit výsledný objekt a reprezentovat tak širokou škálu různých konfigurací. Tímto přístupem se předchází vzniku nabobtnalých seznamů argumentů a zároveň se otevírá prostor pro obohacení jednotlivých kroků nastavení o vstupní logiku a validaci. Builder tak transformuje proces tvorby objektu z jednorázového (často nepřehledného) úkonu na řízenou sekvenci operací.



Obrázek 2.10: Komutativní diagram návrhového vzoru: Builder

Formálně je *Builder* kategorií o třech objektech (*Počátek, Builder, a Product*) a třech morfismech (*inicializace, konfigurace a sestavení*). Počátek reprezentuje stav, kdy *Builder* ješte neexistuje, a je potřeba zavést jeho počáteční konfiguraci.

Morfismus konstruktoru z počátku vytvoří *Builder*, který lze následně konfigurovat pomocí automorfismu konfigurace

Singleton

Facade

Proxy

Composite

Chain of Responsibility

Strategy

Observer

2.3.3 Kompozitní vzory

2.4 ARCHITEKTONICKÉ VZORY

2.5 INTEGRAČNÍ VZORY

2.6 TITLE

Úroveň C4	Název úrovně	Princip řízení kvality	Kapitola
Úroveň 4	Systémový kontext	Integrační vzory	2.5
Úroveň 3	Kontejnery	Architektonické vzory	2.4
Úroveň 2	Komponenty	Návrhové vzory	2.3
Úroveň 1	Kód	Čistý kód	2.2

Tabulka 2.2: Propojení úrovní modelu C4 s principy softwarového inženýrství

3 UDRŽITELNOST PRODUKČNÍHO KÓDU

3.1 TESTOVÁNÍ

3.2 VERZOVÁNÍ

3.3 POSTUPNÁ INTEGRACE

3.4 POSTUPNÉ NASAZENÍ

3.5 MONITORING

Kritickou součástí vývoje softwaru je jeho testování a ladění (debugging). Testy, například jednotkové nebo integrační, simulují chování systému za různých podmínek a následné ladění pomocí debuggerů odhaluje jeho vnitřní stav instrukce po instrukci. V praxi ovšem není možné odhalit všechny chyby a dojde na situace, kdy se v nasazeném řešení chyba objeví. Při lokálním vývoji by se program odkrokoval debuggerem a zjistili se okolnosti, které chybu způsobily.

V produkčním nasazení ovšem luxus odkrokování programu zpravidla nemáme, ať už kvůli chybějícím pomocným strukturám (například *symbol table*) a nebo kvůli optimalizacím kompilátoru, který program převede do jazyka symbolických adres či byte kódu. Když tedy nelze pozorovat stav programu zevnitř, nezbývá nám nic jiného, než ho pozorovat zvenčí - a konkrétně přes jeho výstupy. Schopnost pozorovat chování systému na základě jeho výstupů nazýváme **observabilitou**.

Mnoho autorů a spolků, kteří se observabilitou zabývají, jí rozdělují na pilíře. Cílem těchto pilířů je zodpovědět na otázky: *Co se to děje?*, *Proč se to děje?* a *Kde se to děje?*. Každý pilíř tak doplňuje informace k těm dalším, a dohromady tvoří kompletní vhled do chování systému. (Elastic Observability Team, 2024) Tato práce ovšem bude na observabilitu nahlížet trochu jinak. Paradigma pilířů indikuje, že každý pilíř je disjunktním článkem, které, když se spojí dohromady, tvoří observabilitu.

3.5.1 Logy

Jedním z nejčastějších výstupů programu, který se používá pro ladění nasazeného řešení, jsou logy. Jedná se o textové záznamy, které nesou informaci o událostech v programu. Logování má své zásady a doporučení, jejichž dodržení či nedodržení ovlivňuje míru, kterou nám logy pomohou s diagnostikou problému. Ačkoliv je logování velmi užitečné, nedává nám komplexní obraz o chování systému, protože se jedná jenom o jeden z možných výstupů programu. Program může vytvářet další telemetrická data, která mohou být následně dále zpracována.

3.5.2 Metriky

Metriky jsou kvantitativní veličiny, které měří konkrétní vlastnosti běžícího programu. Můžeme tak měřit například počet požadavků na konkrétní instanci či počet I/O operací. Metriky máme přímo měřené a odvozené, přičemž odvozené metriky můžeme zjišťovat z logů (například počet chyb v čase)

Je nutné podotknout, že metriky má smysl zaznamenávat dlouhodobě jakožto časové řady, abychom určili chování systému v čase. Je nutné tak metriky zaznamenávat na základě události, nebo pravidelného měření. Naopak zaznamenávat metriky vycházející ze statické analýzy smysl nedává, protože pokud bychom chtěli například analyzovat složitost programu pomocí Halsteadových vzorců, tak se tato metrika nemůže měnit v čase běhu programu. Naopak zatížení konkrétní instance programu nebo využití hardwarových prostředků proměnlivé čase běhu programu je.

4 AUDIT ARCHITEKTURY SYSTÉMU

Tato kapitola se zabývá analýzou existujícího řešení. Cílem je stávající řešení zmapovat, modelovat pomocí diagramů, a popisovat ho na dostatečně detailních úrovní abstrakce, která mi dovolí poukázat na přítomný technický dluh. K zmapování a rozdělení na úrovně abstrakce jsem využil C4 diagramy, které navrhnul Simon Brown k popisu architektury na celkem čtyřech úrovních abstrakce (Brown, 2024b).

Ve světe softwarového inženýrství je již zavadený standard, který slouží k modelování softwarové architektury, a to je *Unified Modelling Language*. Ten slouží k modelování a dokumentaci jednotlivých částí a procesů. Modeluje tak jak vztahy mezi třídami, tak zároveň umožňuje zaznamenat komunikaci mezi komponentami pomocí sekvenčního diagramu či cestu uživatele pomocí user journey diagramu (Špánek, 2022). Proč jsem tedy nezvolil tento standard?

Jak popisuje Simon Brown v úvodu do C4 diagramů, týmy očekávají jednoduchost a srozumitelnost zápisu (Brown, 2024b). UML diagramy slouží zároveň jako dokumentace, což je jeho síla, a zároveň jeho slabina. Dokumentace musí být dostatečně podrobná a jednoznačná, s čímž přichází určitá dávka složitosti zápisu. Diagramy se stávají více deskriptivní a více nahuštěné, a zároveň se vytváří diagramy pro každý aspekt softwarového produktu. Množství diagramů tak rapidně stoupá a jejich udržování se stává časově i zdrojově náročnější. Pokud navíc dodržujeme netriviální notaci (například symboly panáčků nebo obdélníků s lištami a popisky), je o něco časově náročnější dokreslit nový blok nebo ho najít v knihovně komponent.

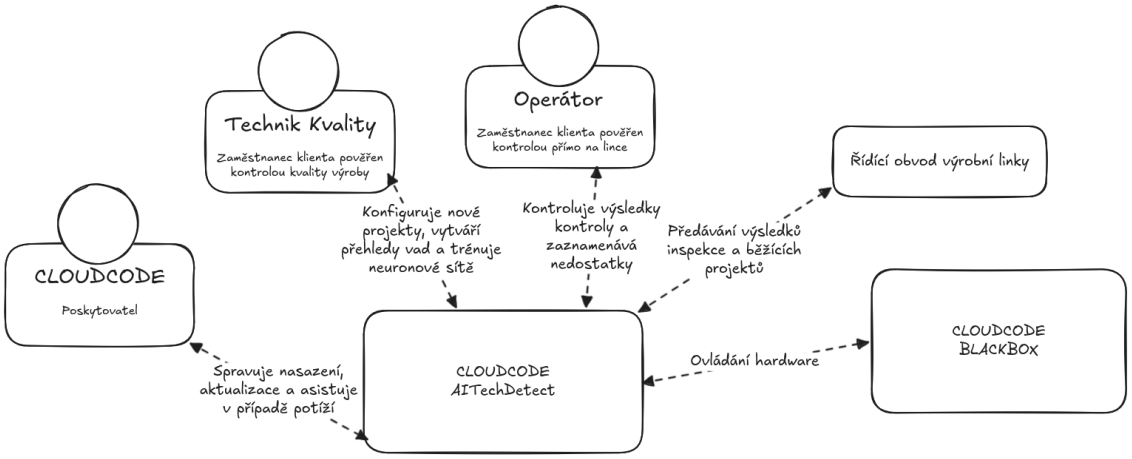
C4 byl navržen právě proto, aby se předešlo složité symbolice a velkému počtu diagramů. Každý diagram je de facto blokový diagram, který se skládá z jednoduchých tvarů a šipek. UML produkuje velké množství diagramů pokrývající každý aspekt zvlášť, zatímco C4 produkuje čtyři diagramy, které ale popisují řešení vždy v celku.

4.1 MODELOVÁNÍ SOFTWAROVÉ ARCHITEKTURY

C4 diagramy dělí softwarovou architekturu na vrstvu systémového kontextu, kontejnerů, komponent a kódu (Brown, 2024b). Diagram lze sestrojit pro každou úroveň zvlášť, přičemž diagram komponent a kódu se dají následně dělit podle jednotlivých kontejnerů a respektive komponent.

4.1.1 Úroveň systémového kontextu

Prvním krokem analýzy je zmapování systému v jeho přirozeném prostředí a přesné vymezení hranic mezi stávajícím řešením a okolními entitami. Tento systémový kontext slouží k vizualizaci obousměrných závislostí: od řešení směrem k externím službám i naopak. Takový pohled umožňuje posoudit kritičnost systému v širším technologickém ekosystému a identifikovat potenciální rizika plynoucí z vnějších vazeb (Brown, 2024c).



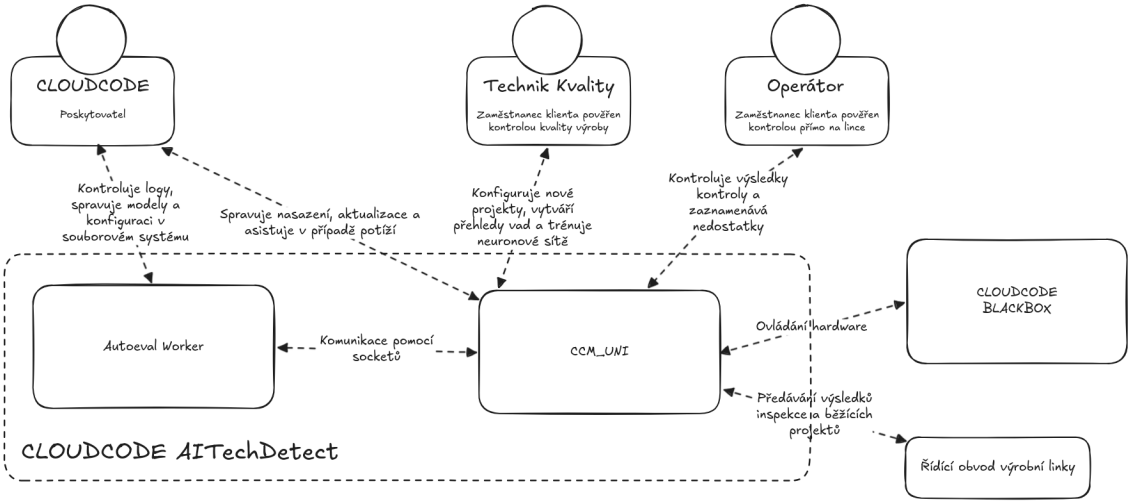
Obrázek 4.1: Diagram systémového kontextu

AITechDetect je koncipován jako izolovaná instalace přímo na hostitelský operační systém. Zatímco dřívější nasazení probíhala na kombinaci distribucí Linux (primárně Ubuntu LTS) a Windows 10, současným standardem je výhradně platforma Windows 11. Tento přechod byl motivován především obavami ze stability ovladačů grafických karet NVIDIA a obecně širší podpory na systémech Windows.

Už z podstaty řešení je jasné, že bude existovat v prostředí výrobní linky, a proto bude s největší pravděpodobností komunikovat s počítači PLC a dalšími nasazenými systémy, jako je například MES. Co se dotčených uživatelů týče, pohybuje se okruh lidí zpravidla ve čtyřech kategoriích: Operátor kvality, neboli dělník na výrobní lince; Mistr výroby, neboli nadřízený operátorům; Technik kvality, který zajišťuje politiku řízení kvality; a zaměstanci CLOUDCODE, kteří nasazené řešení vzdáleně spravují.

4.1.2 Úroveň kontejnerů

Pokud přiblížíme pohled na celkový systém, získáme úroveň kontejnerů. Cílem tohoto diagramu je ukázat, jaké jednotlivé části v systému spolupracují, jak spolu komunikují, a jaké mají odpovědnosti (Brown, 2024a).



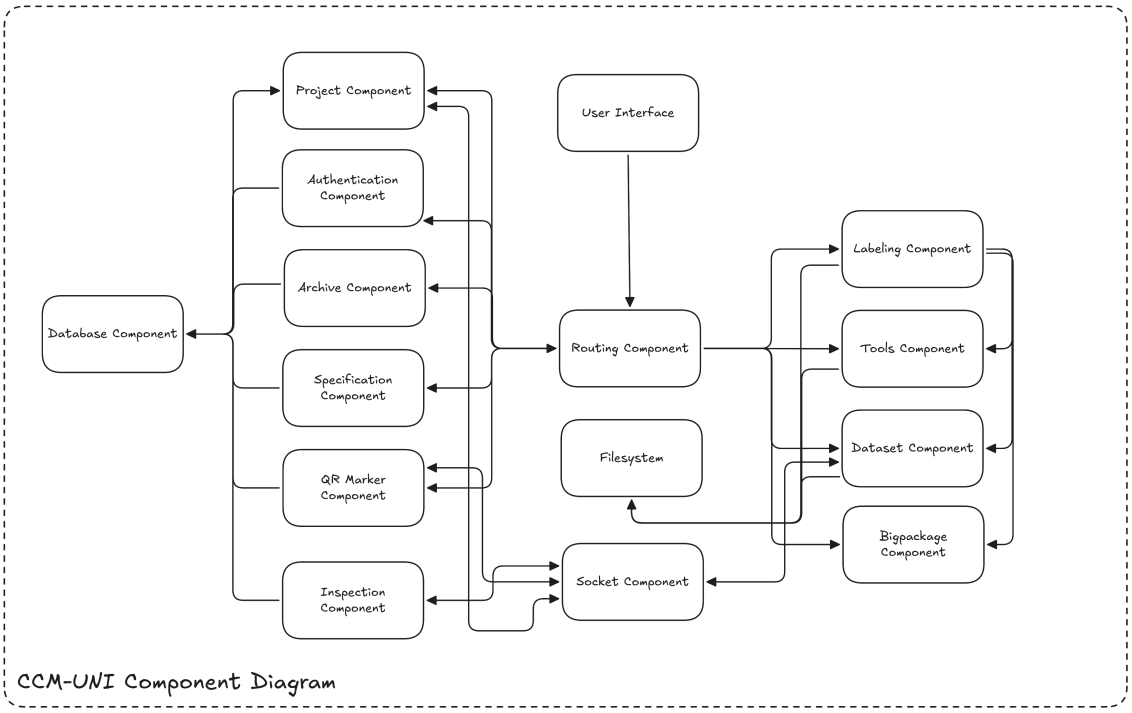
Obrázek 4.2: Diagram komponent

V případě AITechDetect ani nelze tak mluvit o úrovni kontejnerů, když není řešení kontejnerizované, jako o úrovni procesů. Ve standardní distribuci nalezneme dva běžící procesy - hlavní aplikační jádro, které nese název *ccm-uni*, a podpůrnou aplikaci *autoeval*, která vznikla v rámci mé bakalářské práce. Hlavní jádro obstarává vyřizování webových požadavků, práci s databází, manipulaci se strukturovanými daty na disku, a obecně i vstupně výstupní operace. S druhým procesem, Autoevalem, komunikují pomocí socketů, respektive websocketů.

Z hlediska uživatelského užití již můžeme z diagramu pozorovat první rozkol. Zaměstnanci výrobní linky využívají výhradně rozhraní poskytované hlavním aplikačním jádrem, zatímco zaměstanci CLOUDCODE musí při správě a údržbě zasahovat i do procesu Autoeval, které žádné rozhraní neposkytuje. Tato část údržby bude souviset s nečekanými pády či výjimkami, které neodhalil vnitřní testovací proces.

Tím, že komunikace probíhá přes websockety, tak nelze garantovat spolehlivost přenosu.

4.1.3 Úroveň komponent



Obrázek 4.3: Diagram kontejnerů

4.2 MODELOVÁNÍ DOMÉNOVÝCH PROCESŮ

4.3 MODELOVÁNÍ PROCESU NASAZENÍ

4.4 VYHODNOCENÍ RIZIK

Po dokončení modelování softwarové architektury a doménových procesů vyvstala potřeba efektivně ohodnotit oblasti zatížené technickým dluhem. Mým cílem bylo nalézt metriku, která by minimalizovala subjektivní posouzení a umožnila rizika vyplývající z technického dluhu kvantifikovat. Právě soustředění se na riziko mě přivedlo k metodě FMEA (Failure Mode and Effects Analysis), které se věnoval ve své kvalifikační práci můj dlouholetý kolega z Univerzity Jana Evangelisty Purkyně v Ustí nad Labem, Bc. Matěj Vokál.

Metoda FMEA, etablovaná především v automobilovém průmyslu pro detekci vad při návrhu a produkci dílů, se nezaměřuje pouze na izolované nedostatky, ale na identifikaci kritických bodů selhání systému (Vokál, 2025). Tento analytický přístup jsem adaptoval pro potřeby

zkoumaného softwarového řešení, neboť mi umožňuje hodnotit technický dluh prostřednictvím jasně definovaných kritérií.

Výstupem analýzy FMEA je matice, v níž každý řádek reprezentuje konkrétní systémovou položku a její potenciální způsob selhání (Failure Mode). Pro každý identifikovaný bod selhání je následně zpracována analýza vlivů, která postihuje jak bezprostřední lokální dopady v rámci dané komponenty, tak šíření poruchy do vyšších úrovní architektury a její celkový vliv na systém. Samotná kvantifikace rizik probíhá prostřednictvím definovaných hodnotících parametrů, mezi které patří pravděpodobnost výskytu, míra závažnosti, obtížnost detekce a průměrná doba do odhalení selhání. Z těchto faktorů je následně kalkulováno výsledné riziko, které slouží jako objektivní podklad pro prioritizaci a plánování následných nápravných opatření (Vokál, 2025).

Právě komplexní popis šíření selhání systémem a možnost objektivní kvantifikace rizik byly hlavními důvody, proč jsem si metodu FMEA vybral pro analýzu stávajícího řešení společnosti CLOUDCODE. Aby bylo hodnocení technického dluhu v rámci systému CLOUDCODE objektivní, adaptoval jsem standardní desetistupňové škály FMEA na specifické podmínky vývoje softwaru. Každý identifikovaný nedostatek (např. porušení návrhového vzoru nebo vysoký počet parametrů funkce) je posuzován z hlediska následujících faktorů:

- **Vážnost selhání** (S - Severity)
- **Výskyt selhání** (O - Occurrence)
- **Detekce selhání** (D - Detekce)

Tabulka 4.1: Škála vážnosti technického dluhu

Stup.	**Význam**	Zotavení	Debugovatelnost
1	**Bez dopadu**.	Ano	Žádný vliv na spolehlivost a rozšířitelnost systému.
2	**Velmi nízká**.	Ano	
3	**Nízká**.	Ano	
4	**Střední**.	Ano	Produkt je lehce poškozen. Zpozoruje běžný zákazník.
5	**Vysoká**.	Ano	Produkt je lehce poškozen. Zpozoruje běžný zákazník.
6	**Kritická**.	Ano	Systém se umí ze selhání zotavit, ale výskyt selhání negativně ovliní jeho běh.
7	**Katastrofální**.	Ne	ystém se stává nefunkčním a je obtížné dotrasovat příčinu selhání.

Stup.	**1**	**2**	**3**	**4**	**5**	**6**
1	**Jistá**.					
2	**Skoro jistá**.					
3	**Vysoká**.					
4	**Průměrná**.					
5	**Nízká**.					
6	**Nedetekovatelná**					

Tabulka 4.2: Matice rizik

5 ARCHITEKTURA NOVÉHO ŘEŠENÍ

5.1 POSTUPNÉ NASAZENÍ

6 ZÁVĚR

BIBLIOGRAFIE

BROWN, Simon, 2024a. *Container diagram* [online]. 2024-08-14. [cit. 2026-03-13]. Dostupné z: https://c4model.com/diagrams/container.

BROWN, Simon, 2024b. *Introduction* [online]. 2024-08-14. [cit. 2026-03-13]. Dostupné z: https://c4model.com/introduction.

BROWN, Simon, 2024c. *System context diagram* [online]. 2024-08-14. [cit. 2026-03-13]. Dostupné z: https://c4model.com/diagrams/system-context.

DANĚK, Kevin, 2024. *Rapidní vývoj algoritmů strojového vidění* [online]. 2024-06-11. [cit. 2026-03-02]. Dostupné z: https://dspace.tul.cz/handle/15240/175790.

ELASTIC OBSERVABILITY TEAM, 2024. *The 3 pillars of observability: Unified logs, metrics, and traces* [online]. 2024-10-10. [cit. 2025-08-09]. Dostupné z: https://www.elastic.co/blog/3-pillars-of-observability.

FORRESTKNIGHT, 2025. *Everything You NEED to Know About TECH DEBT* [online]. 2025-07-24. [cit. 2025-08-10]. Dostupné z: https://www.youtube.com/watch?v=ukgmp6uxQJc.

FOWLER, Martin, 2009. *Technical Debt Quadrant* [online]. 2009-10-14. [cit. 2025-08-10]. Dostupné z: https://martinfowler.com/bliki/TechnicalDebtQuadrant.html.

MA, Noah, 2026. *Category* [online]. 2026-04-04. [cit. 2026-04-04]. Dostupné z: https://docs.racket-lang.org/ctp/_Category_.html.

MÄNTYLÄ, Mika, 2010. Empirical software evolvability - Code smells and human evaluations. In: s. 1–6. Dostupné z DOI: 10.1109/ICSM.2010.5609545.

MARTIN, Robert C.; COPLIEN, James O., 2009. *Clean code: a handbook of agile software craftsmanship*. Prentice Hall. ISBN 9780132350884.

MICHIGAN TECHNOLOGICAL UNIVERSITY, 2025. *What is software engineering?* [online]. 2025-04-09. [cit. 2026-03-03]. Dostupné z: https://www.mtu.edu/cs/undergraduate/software/what/.

MIKKONEN, Tommi, 1998. Formalizing design patterns. In: *Proceedings of the 20th international conference on Software engineering*. IEEE, s. 115–124.

NIERSTRASZ, Oscar, 1992. Towards an Object Calculus. In: ISBN 978-3-540-55613-8. Dostupné z DOI: 10.1007/3-540-55613-3_1.

PEREZ, Javier, 2011. *Refactoring Planning for Design Smell Correction in Object-Oriented Software*. Dis. pr.

SHVETS, Alexander, 2018a. *Code Smells* [online]. 2018-02-08. [cit. 2026-03-03]. Dostupné z: https://refactoring.guru/refactoring/smells.

SHVETS, Alexander, 2018b. *Code Smells* [online]. 2018-02-08. [cit. 2026-03-03]. Dostupné z: https://refactoring.guru/design-patterns/what-is-pattern.

SONARSOURCE, 2025. *Measuring and Identifying Code-level Technical Debt: A Practical Guide* [online]. 2025-10-21. [cit. 2026-03-25]. Dostupné z: https://www.sonarsource.com/resources/library/measuring-and-identifying-code-level-technical-debt-a-practical-guide/.

ŠPÁNEK, Roman, 2022. *Přednáška č. 5: UML I.* [Přednáška v rámci předmětu Softwarové inženýrství]. [cit. 2026-03-13]. Technická univerzita v Liberci.

TEAM ASANA, 2025. *What is technical debt? How to pay it off (with examples)* [online]. 2025-06-25. [cit. 2025-08-10]. Dostupné z: https://asana.com/resources/technical-debt.

TORVALDS, Linus, 2007. *Re: Coding Style* [online]. 2007-06-15. [cit. 2026-03-03]. Dostupné z: https://yarchive.net/comp/linux/coding_style.html.

VOKÁL, Matěj, 2025. *Zpracování harmonizované FMEA pro kryt a podlahu zavazadlového prostoru vozu Škoda Elroq dodávaného IMG AUTOMOTIVE s.r.o.* [online]. [cit. 2026-03-14]. Bakalářská práce. Univerzita Jana Evangelisty Purkyně. Fakulta strojního inženýrství. Ústí nad Labem.

A ADAPTOVANÁ HODNOTÍCÍ ŠKÁLA VÁŽNOSTI SELHÁNÍ

Stup.	Význam
1	**Bez dopadu**. Žádný vliv na spolehlivost a rozšířitelnost systému.
2	**Velmi nízká**. Produkt Zpozoruje pouze diskriminující zákazník.
3	**Nízká**. Produkt je lehce poškozen. Zpozoruje běžný zákazník.
4	**Střední**. Produkt je lehce poškozen. Zpozoruje běžný zákazník.
5	**Vysoká**. Produkt je lehce poškozen. Zpozoruje běžný zákazník.
6	**Kritická**. Systém se umí ze selhání zotavit, ale výskyt selhání negativně ovliní jeho běh.
7	**Katastrofální**. Systém se stává nefunkčním a je obtížné dotrasovat příčinu selhání.

B ADAPTOVANÁ HODNOTÍCÍ ŠKÁLA VÝSKYTU SELHÁNÍ

Stup.	Definice

C ADAPTOVANÁ HODNOTÍCÍ ŠKÁLA DETEKCE SELHÁNÍ

Stup.	Definice